

GCC Built-in Functions for Atomic Memory Access

These built-in functions are designed for compatibility with the Intel Itanium Processor-specific Application Binary Interface (ABI), section 7.4. Unlike standard GCC practices, these functions do not utilize the `__builtin_` prefix and are overloaded to support multiple data types.

Overview and Type Support

While the original Intel documentation limits these operations to `int`, `long`, and `long long` (and their unsigned counterparts), GCC extends this support. GCC permits any integral scalar or pointer type with a length of 1, 2, 4, or 8 bytes.

Target Processor Limitations

Support for these operations varies by target processor. If an operation is not supported by the hardware:

- A warning will be generated during compilation.
- The compiler will generate a call to an external function.
- These external functions follow the naming convention of the built-in with an additional suffix `_n`, where `n` represents the size of the data type.

Memory Barrier Semantics

In most instances, these built-ins act as a **full barrier**. This ensures that no memory operand is moved across the operation in either direction. Additionally, instructions are issued to prevent the processor from speculating loads across the operation or queuing stores after it.

Regarding the Intel documentation's mention of an "optional list of variables protected by the memory barrier," GCC currently ignores this list and protects all globally accessible variables.

Atomic Operations: Fetch and Op

These routines perform the specified operation and return the value that was in memory **prior** to the operation.

Built-in Function	Logic Summary
<code>type __sync_fetch_and_add (type *ptr, type value, ...)</code>	<code>{ tmp = *ptr; *ptr += value; return tmp; }</code>
<code>type __sync_fetch_and_sub (type *ptr, type value, ...)</code>	<code>{ tmp = *ptr; *ptr -= value; return tmp; }</code>
<code>type __sync_fetch_and_or (type *ptr, type value, ...)</code>	<code>{ tmp = *ptr; *ptr = value; return tmp; }</code>
<code>type __sync_fetch_and_and (type *ptr, type value, ...)</code>	<code>{ tmp = *ptr; *ptr &= value; return tmp; }</code>
<code>type __sync_fetch_and_xor (type *ptr, type value, ...)</code>	<code>{ tmp = *ptr; *ptr ^= value; return tmp; }</code>
<code>type __sync_fetch_and_nand (type *ptr, type value, ...)</code>	<code>{ tmp = *ptr; *ptr = ~tmp & value; return tmp; }</code>

Atomic Operations: Op and Fetch

These routines perform the specified operation and return the **new** value currently in memory.

Built-in Function	Logic Summary
<code>type __sync_add_and_fetch (type *ptr, type value, ...)</code>	<code>{ *ptr += value; return *ptr; }</code>

Built-in Function	Logic Summary
<code>type __sync_sub_and_fetch (type *ptr, type value, ...)</code>	<code>{ *ptr -= value; return *ptr; }</code>
<code>type __sync_or_and_fetch (type *ptr, type value, ...)</code>	<code>{ *ptr = value; return *ptr; }</code>
<code>type __sync_and_and_fetch (type *ptr, type value, ...)</code>	<code>{ *ptr &= value; return *ptr; }</code>
<code>type __sync_xor_and_fetch (type *ptr, type value, ...)</code>	<code>{ *ptr ^= value; return *ptr; }</code>
<code>type __sync_nand_and_fetch (type *ptr, type value, ...)</code>	<code>{ *ptr = ~*ptr & value; return *ptr; }</code>

Compare and Swap (CAS)

These functions perform an atomic compare and swap. If the value at `*ptr` matches `oldval`, then `newval` is written to `*ptr`.

- `bool __sync_bool_compare_and_swap (type *ptr, type oldval, type newval, ...)`: Returns `true` if the comparison was successful and the new value was written.
- `type __sync_val_compare_and_swap (type *ptr, type oldval, type newval, ...)`: Returns the contents of `*ptr` before the operation.

Additional Synchronization Functions

Full Synchronization

- `__sync_synchronize (...)`: Issues a full memory barrier.

Legacy Lock Support

- **type __sync_lock_test_and_set (type *ptr, type value, ...):**
This is an atomic exchange operation rather than a traditional test-and-set. It writes **value** to ***ptr** and returns the previous content.
 - *Note:* On targets with minimal lock support, this may only support storing the constant 1.
 - *Barrier Type:* This is an **acquire barrier**. Memory references after the built-in cannot move before it, but previous stores may not yet be globally visible.
- **void __sync_lock_release (type *ptr, ...):** Releases a lock by writing 0 to ***ptr**.
 - *Barrier Type:* This is a **release barrier**. All previous memory operations are made visible/satisfied, but subsequent reads are not prevented from being speculated before the barrier.